Лабораторная работа №10

Тема: Программирование экспорта данных из БД

Цель: получить практические навыки формирования и вывода результатных данных в форматах XLSX, HTML средствами MS Visual Studio.

Объем: 4 академических часа (1 неделя).

Задание

1. На основе приложения, разработанного при выполнении лабораторных работ №8 и №9, создать форму для обеспечения доступа к результатным данным и их экспорта в Excel и HTML. Созданную форму связать с соответствующим пунктом меню «Экспорт» главной формы приложения. В форме обеспечить:
   - доступность разработанных при выполнении п.4 задания к лабораторной работе №6 представлений (View) для формирования выходных данных посредством запросов к базе данных Postgre SQL;
   - выбор выходного документа посредством CheckBox;
   - механизм сохранения книги Excel и файла HTML посредством SaveFileDialog;
   - выбор операции экспорта посредством ComboBox либо GroupBox с двумя RadioButton;
   - реализацию операций экспорта Excel и HTML в обработчике события нажатия (Click) кнопки (Button);
   - обработку исключений на всех этапах вычислений.
2. Посредством интерфейсных средств приложения обеспечить ввод в БД разнообразных данных, достаточных для тестирования операций экспорта результатных инфопотоков.
3. Выполнить пользовательское тестирование операций экспорта и убедиться в соответствии полученных результатов потребностям пользователей.

Методические указания к выполнению

Разработанный макет формы Export для выполнения операций экспорта данных показан на рис. 10.1. На него вынесены все необходимые элементы управления, в том числе флаги для выбора необходимого представления (CheckBox) и элементы для выбора способа экспорта (RadioButton), а также механизм сохранения файлов (SaveFileDialog). Указанные здесь элементы добавлены простым перетаскиванием из окна «Панель элементов» при помощи мыши и никаких настроек не требуют.

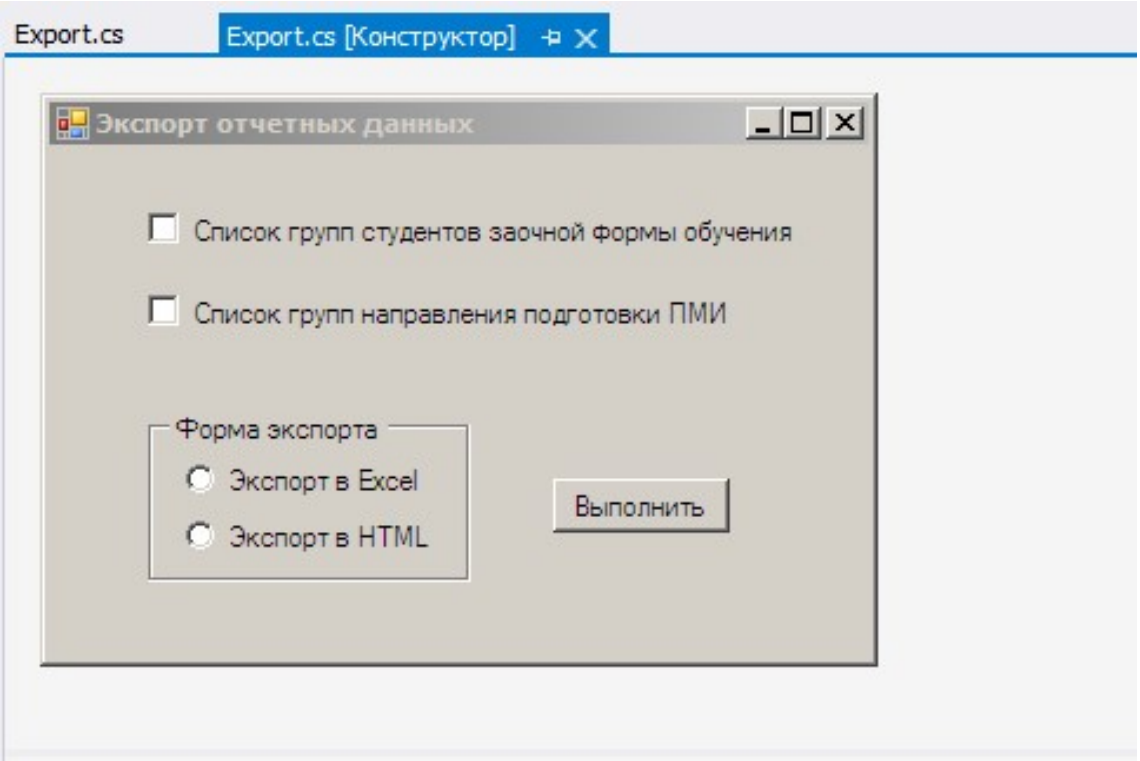


Рисунок 10.1 – Макет формы для выполнения операций экспорта

Создание экспорта в Excel

Для выполнения экспорта в Excel разработана функция `ExportToExcel` с параметром `name`, который инициализируется именем представления или таблицы, соответствующее отмеченному в форме выходному потоку:

```
void ExportToExcel(string name)
    {
      List<List<string>> table = PG.Select_all(name);
      saveFileDialog1.FileName = name;
```

```
        if (saveFileDialog1.ShowDialog() == DialogResult.OK)
        {
            FileStream Stream1 = new FileStream(saveFileDialog1.FileName, FileMode.Create);
            try
            {
                StreamWriter StreamWriterStream1 = new StreamWriter(Stream1, Encoding.Unicode);

                foreach (string Column in table[0])
                    StreamWriterStream1.Write(Column + "\t");

                StreamWriterStream1.WriteLine();
                bool skip1 = true;
                foreach (List<string> Row in table)
                {
                    if (skip1) skip1 = false;
                    else
                    {
                        foreach (object Entity in Row)
                        {
                            StreamWriterStream1.Write(Entity.ToString() + "\t");
                        }
                        StreamWriterStream1.WriteLine();
                    }
                }
                StreamWriterStream1.Flush();
            }
            catch
            {
                MessageBox.Show("При передаче данных возникла ошибка!", "Ошибка!",
MessageBoxButtons.OK, MessageBoxIcon.Error);
                return;
            }
            Stream1.Close();
            Process.Start(Stream1.Name);
        }
    }
```

В этой функции компонент saveFileDialog обеспечивает открытие стандартного диалогового окна сохранения с возможностью выбора места и введения имени. Для потокового вывода (FileStream и т.д.) нужно подключить библиотеку `System.IO`. Для запуска процесса в конце тела функции нужно подключить библиотеку `System.Diagnostics`. Для поддержки запуска стандартных Windows модальных диалогов, необходимо в файле Program.cs для функции Main установить атрибут [STAThread], если в проекте он не настроен по умолчанию.

При вызове функции `ExportToExcel` осуществляется запрос к серверу с помощью метода Select_all класса PG, пример которого рассматривался в лабораторных №8 и №9.

Этот вызов полезно предварить командами настройки компонента saveFileDialog на работу с книгой Excel:

```
saveFileDialog1.DefaultExt = "xls";
saveFileDialog1.Filter = "Excel files(*.xls)|*.xls";
saveFileDialog1.Title = "Экспорт: Excel";
```

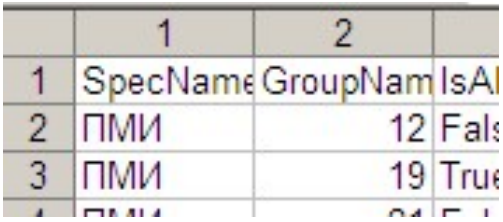
Создание экспорта в HTML

Функция для выполнения экспорта в HTML работает похожим образом, при этом имя запрашиваемой таблицы передается ещё и как подпись в заголовок страницы. В теле функции страница размечается и раскрашивается на языке HTML.

```
void ExportToHTML(string name)
    {
      List<List<string>> table = PG.Select_all(name);
      saveFileDialog1.FileName = name;

      int i, j;

      if (saveFileDialog1.ShowDialog() == System.Windows.Forms.DialogResult.OK)
      {
        FileStream Stream1 = new FileStream(saveFileDialog1.FileName, FileMode.Create);
        try
        {
          StreamWriter StreamWriter1 = new StreamWriter(Stream1);

          StreamWriter1.WriteLine("<html>");
          StreamWriter1.WriteLine("<head>");
          StreamWriter1.WriteLine("<meta content=\"text/html; charset=utf-8\" http-equiv=\"Content-
Type\">");
          StreamWriter1.WriteLine("<title>" + name + "</title>");
          StreamWriter1.WriteLine("</head>");
          StreamWriter1.WriteLine("<body bgcolor=\"800000\">");
          StreamWriter1.WriteLine("<table align=\"center\" cols =0 cellspacing =0>");
          StreamWriter1.WriteLine("<tr>");
          StreamWriter1.WriteLine("</td>");
          StreamWriter1.WriteLine("</tr>");
          StreamWriter1.WriteLine("<tr>");
          for (j = 0; j < table[0].Count; j++)
          {
            StreamWriter1.WriteLine("<td><font face=\"Verdana\"size=\"2\" color=\"#ffffff\"><p
align=\"center\"><b>");
            StreamWriter1.WriteLine("" + table[0][j]);
            StreamWriter1.WriteLine("</b></font></td>");
          }
          StreamWriter1.WriteLine("</tr>");
          for (i = 1; i < table.Count; i++)
          {
            if ((i % 2 - 1) == 0)
```

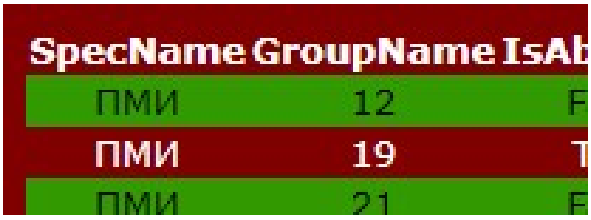
```
                {
                    StreamWriter1.WriteLine("<tr bgcolor=\"3399\">");
                    for (j = 0; j < table[i].Count; j++)
                    {
                        StreamWriter1.WriteLine("<td><font face=\"Verdana\"size=\"2\"
color=\"#000000\"><p align=\"center\">");
                        StreamWriter1.WriteLine("" + table[i][j]);
                        StreamWriter1.WriteLine("</font></td>");
                    }
                    StreamWriter1.WriteLine("</tr>");
                }
                else
                {
                    StreamWriter1.WriteLine("<tr>");
                    for (j = 0; j < table[i].Count; j++)
                    {
                        StreamWriter1.WriteLine("<td><font face=\"Verdana\"size=\"2\"
color=\"#ffffff\"><p align=\"center\">");
                        StreamWriter1.WriteLine("" + table[i][j]);
                        StreamWriter1.WriteLine("</font></td>");
                    }
                    StreamWriter1.WriteLine("</tr>");
                }
            }
            StreamWriter1.WriteLine("</table></center></body></html>");
            StreamWriter1.Flush();
        }
        catch
        {
            MessageBox.Show("При передаче данных возникла ошибка!", "Ошибка!",
MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
        Stream1.Close();
        Process.Start(Stream1.Name);
    }
}
```

Результаты экспорта показаны на рис. 10.3.



	1	2	
1	SpecName	GroupNam	IsA
2	ПМИ	12	Fals
3	ПМИ	19	True

а) экспорт в Excel



SpecName	GroupName	IsAb
ПМИ	12	F
ПМИ	19	T
ПМИ	21	F

а) экспорт в HTML

Рисунок 10.3 – Результаты экспорта данных из БД

Все методы экспорта должны вызываться при нажатии одной кнопки, при этом количество и тип вызовов должны зависеть от состояний флагов на форме.

Пример обработки события нажатия кнопки:

```
private void button1_Click(object sender, EventArgs e)
    {
      if (radioButton1.Checked)
      {
        saveFileDialog1.DefaultExt = "html";
        saveFileDialog1.Filter = "Hyper text|*.html";
        saveFileDialog1.Title = "Export HTML";
        if (checkBox1.Checked) ExportToHTML("specialities");
        if (checkBox2.Checked) ExportToHTML("sets");
      }
      if (radioButton2.Checked)
      {
        saveFileDialog1.DefaultExt = "xls";
        saveFileDialog1.Filter = "Excel|*.xls";
        saveFileDialog1.Title = "Export Excel";
        if (checkBox1.Checked) ExportToExcel("specialities");
        if (checkBox2.Checked) ExportToExcel("sets");
      }
    }
```

Требования к содержанию отчета

В отчет необходимо включить:

- команды SELECT представлений выходных данных;
- изображение разработанной формы в режиме запуска;
- листинг модуля разработанной формы;
- результаты экспорта в Excel и HTML.